

AI Ops – Assignment 2

Sri Harsha V
September 2026

Question 1

Image Sizes

The naive single-stage image was built using `python:3.10`, while the multi-stage image used `python:3.10` as the builder stage and `python:3.10-slim` as the runtime stage.

Image	Disk Usage	Content Size
spam-api-naive	2.08 GB	515 MB
spam-api-multistage	676 MB	154 MB

Table 1: Measured image sizes.

The percentage reduction in content size is

$$\frac{515 - 154}{515} \times 100 = 70.1\%.$$

The percentage reduction in disk usage is

$$\frac{2.08 - 0.676}{2.08} \times 100 = 67.5\%.$$

Why the Multi-Stage Image Is Smaller

The naive image uses the larger `python:3.10` image as its final runtime base, whereas the multi-stage image uses the smaller `python:3.10-slim` image for the final runtime stage. The measured content sizes of these base images were 424 MB and 50.2 MB, respectively.

In the multi-stage build, Python dependencies are installed in the `python:3.10` builder stage. The final `python:3.10-slim` stage then receives only the installed dependencies required by the application and the application files. The builder image and its intermediate build environment are therefore not included in the final runtime image. This reduces the measured content size from 515 MB for the naive image to 154 MB for the multi-stage image, a reduction of 70.1%.

Question-2

Cache Performance

For the same input, the first request took 0.039920s, while the repeated request took 0.004057s, demonstrating a measurable speedup from the Redis cache. The corresponding terminal output is provided in the Q2 evidence folder.

Compose vs. Kubernetes

Docker Compose coordinates multiple related containers as a single application on one machine, providing service definitions, networking, and a convenient way to start the complete stack. Kubernetes is designed for orchestrating workloads across a cluster of multiple nodes, providing scheduling, scaling, and self-healing capabilities.

Question 3

3.1 Parallelism Justification

The cluster has 4 total allocatable CPUs across two nodes, with 2 CPUs available per node. Each validation pod was assigned a CPU request and limit of 500m. In theory, the total CPU capacity can accommodate

$$\frac{4}{0.5} = 8$$

validation pods concurrently. Therefore, `parallelism: 8` was chosen because there are exactly 8 shards, allowing the Job to schedule up to all 8 pods when sufficient CPU capacity is available.

In the actual cluster, some CPU capacity was already reserved by other running workloads. The final resource evidence shows that the `minikube` node had 2 CPUs allocatable, with 1100m already requested, leaving approximately 900m. The `minikube-m02` node also had 2 CPUs allocatable, with 350m already requested, leaving approximately 1650m. With a 500m request per validation pod, this provides capacity for approximately 1 additional pod on `minikube` and 3 additional pods on `minikube-m02`, giving approximately 4 validation pods that can run concurrently under the observed resource conditions.

The remaining validation pods were therefore scheduled as CPU capacity became available. All 8 indexed shard-validation pods eventually completed successfully.

3.2 Evidence

The supporting execution evidence is provided in the `Q3/evidence/` directory. It includes the cluster CPU configuration, Indexed Job execution and pod scheduling across the two nodes, completion of all 8 shard-validation pods, and collection of the validation results through the Kubernetes API.

3.3 Validation Results

The results were collected using the Kubernetes API by reading the logs of the completed Job pods:

```
(.venv) sriharsha@sri-harsha-v:~/projects/DA3408_DA24B034_2/Q3$ python3 collect_results.py
```

Shard	Total Rows	Invalid Rows	Pod	Node
0	50	3	shard-validation-job-0-5psdh	minikube-m02
1	50	5	shard-validation-job-1-m9mft	minikube
2	50	8	shard-validation-job-2-s58jt	minikube-m02
3	50	7	shard-validation-job-3-lcfq9	minikube-m02
4	50	4	shard-validation-job-4-cctb5	minikube-m02
5	50	6	shard-validation-job-5-w9g44	minikube-m02
6	50	9	shard-validation-job-6-cjxfx	minikube-m02
7	50	5	shard-validation-job-7-bcrng	minikube

Total invalid rows across all shards: 47

Saved full results to `shard_results.json`

All 8 shards were successfully validated, with 50 rows processed per shard. The validation identified 47 invalid rows in total, and the results were collected from pods running on both cluster nodes.

A shared volume was avoided because Minikube's default storage provisioner can bind storage to a particular node, while the validation pods may be scheduled on either node. Reading the pod logs through the Kubernetes API therefore provides node-independent access to the results.

3.4 Re-Justification for a 6-CPU Cluster

If the cluster had 6 CPUs instead of 4, with the same 500m CPU request per pod, up to

$$\frac{6}{0.5} = 12$$

pods could theoretically be scheduled concurrently. However, there are only 8 shards, so **parallelism: 8** would still be sufficient. The additional CPU capacity would provide more resource headroom, but would not increase the number of shard-validation pods beyond 8.

Question 4

Self-Healing

The initial state showed two running Pods:

```
$ kubectl get pods -l app=spam-api
```

NAME	READY	STATUS	RESTARTS	AGE
spam-api-deployment-546bd7dd45-46lr8	1/1	Running	0	2m40s
spam-api-deployment-546bd7dd45-gqbt4	1/1	Running	0	2m40s

One Pod was then deleted:

```
$ kubectl delete pod spam-api-deployment-546bd7dd45-46lr8
pod "spam-api-deployment-546bd7dd45-46lr8" deleted from default namespace
```

```
$ kubectl get pods -l app=spam-api
```

NAME	READY	STATUS	RESTARTS	AGE
spam-api-deployment-546bd7dd45-gqbt4	1/1	Running	0	4m2s
spam-api-deployment-546bd7dd45-smc8l	1/1	Running	0	55s

The deleted Pod was automatically replaced by a new Pod within seconds, restoring the desired replica count of 2 without manual intervention. The other original Pod remained untouched. The replacement Pod subsequently reached `1/1 Running`.

The Deployment manages a ReplicaSet, which maintains the desired replica count specified by the Deployment. When the Pod was deleted, the ReplicaSet detected that only one matching Pod remained instead of the desired two and automatically created a replacement Pod.

Rolling Update

The `/healthz` response was changed to include `"version": "v2"`. A new image, `spam-api-multistage:v2`, was built and the Deployment was updated using:

```
kubectl set image deployment/spam-api-deployment \
  spam-api=spam-api-multistage:v2
```

The rollout completed successfully using `kubectl rollout status`, and `kubectl rollout history` showed the new revision. The updated `/healthz` endpoint returned version `v2`, while `/predict` continued to work. The corresponding build, rollout, and API verification evidence is provided in the Q4 evidence directory.

Deployment vs. Job

A Deployment is used for long-running applications because it maintains the desired number of running Pods, provides self-healing, and supports rolling updates. A Job is used for run-to-completion batch work, where Pods perform a fixed task and terminate after completing it.